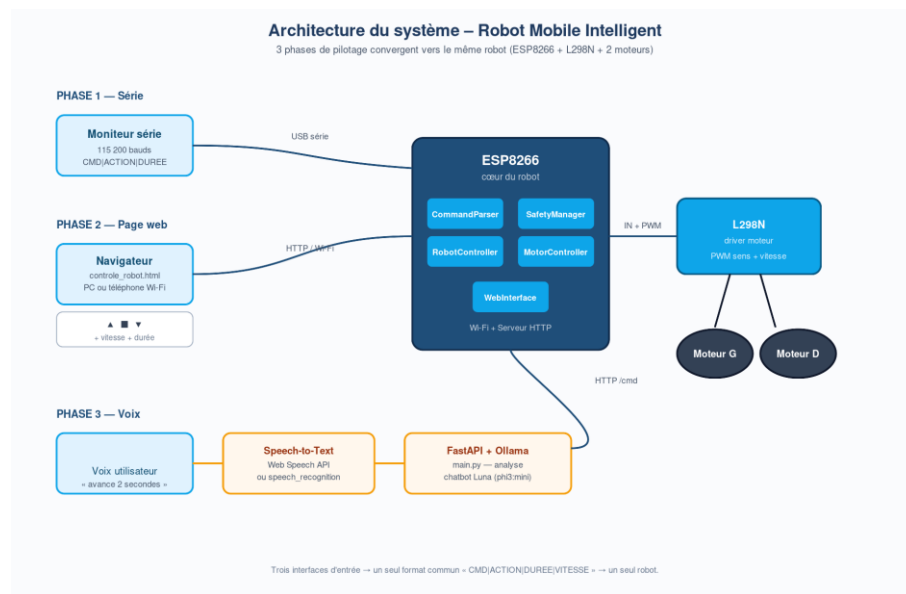


RAPPORT DE PROJET

Robot Mobile Intelligent

Conception, programmation et contrôle vocal d'un robot à base d'ESP32



Projet Arduino — Fin de semestre

Auteurs

Camara Cherif Aboubacar

Magagi Ibrahima Laila

Fadera Seynabou Dembadiang

Elmahouer Chaimae

Niveau

Cycle Préparatoire Intégrée d'EIDIA

Année

2025 — 2026

Sommaire

1. Introduction
2. Cahier des charges et objectifs
3. Architecture générale du système
 - 3.1. Phase 1 — Déplacements simples
 - 3.2. Phase 2 — Interface web embarquée
 - 3.3. Phase 3 — Assistant vocal et chatbot
4. Matériel utilisé et brochage
5. Présentation du code
 - 5.1. Sketch Arduino (ESP32)
 - 5.2. Page de contrôle web
 - 5.3. Backend Python + Ollama
6. Difficultés rencontrées
7. Solutions apportées
8. Résultats obtenus
9. Améliorations possibles
10. Conclusion

1. Introduction

Ce projet a pour objectif de concevoir et de réaliser un robot mobile « intelligent » capable d'être piloté de plusieurs manières complémentaires : par un simple moniteur série, par une interface web hébergée directement sur le microcontrôleur, et enfin par la voix grâce à un assistant équipé d'un modèle de langage local. Le projet est construit autour d'une carte ESP32 (module ESP-WROOM-32), choisie pour son faible coût, son Wi-Fi intégré et sa compatibilité avec l'environnement Arduino.

Le travail a été pensé en trois phases successives, chacune ajoutant une couche de fonctionnalités. La phase 1 consiste à faire bouger le robot avec des commandes texte simples. La phase 2 ajoute une interface web légère hébergée par l'ESP32 lui-même, accessible depuis un téléphone ou un ordinateur connecté au même Wi-Fi. La phase 3 ajoute un serveur Python (FastAPI) couplé à un modèle de langage local (Ollama, modèle phi3:mini) afin que le robot puisse comprendre des phrases en français comme « avance pendant 2 secondes » ou tenir une courte conversation via la persona « Luna ».

2. Cahier des charges et objectifs

Les exigences fonctionnelles et techniques posées au début du projet sont rassemblées dans le tableau ci-dessous. Elles ont guidé l'ensemble des choix techniques et de conception.

Catégorie	Exigence
Mobilité	Avancer, reculer, tourner à gauche, tourner à droite, s'arrêter.
Sécurité	Bornage de la durée (max 2 s) et de la vitesse (80 à 255) sur le robot.
Pilotage local	Via moniteur série au format CMD ACTION DUREE VITESSE.
Pilotage distant	Page web HTML/CSS/JS hébergée par l'ESP32 en Wi-Fi.
Interaction vocale	Reconnaissance vocale en français + chatbot conversationnel.
Modularité du code	Programmation orientée objet, classes responsables d'une seule tâche.
Compatibilité	ESP32 Arduino Core v3.x avec la nouvelle API PWM ledc.
Coût	Composants accessibles : ESP-WROOM-32, L298N, châssis 2 roues.

3. Architecture générale du système

Le projet est structuré en couches successives. Chaque phase a pu être testée indépendamment avant de passer à la suivante, ce qui a beaucoup facilité le débogage.

3.1. Phase 1 — Déplacements simples

L'utilisateur envoie une commande texte au format CMD|ACTION|DUREE_MS|VITESSE au robot via le moniteur série. La carte ESP32 lit la commande, l'analyse via la classe CommandParser, vérifie sa validité via SafetyManager, applique la limite de durée et de vitesse, puis pilote les moteurs par la classe MotorController.

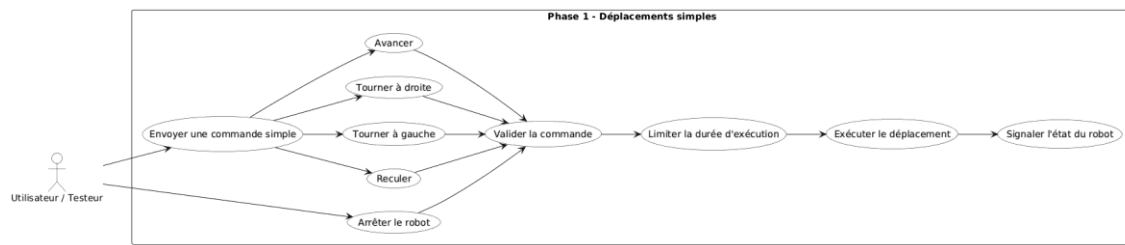


Figure 1 — Cas d'utilisation, phase 1

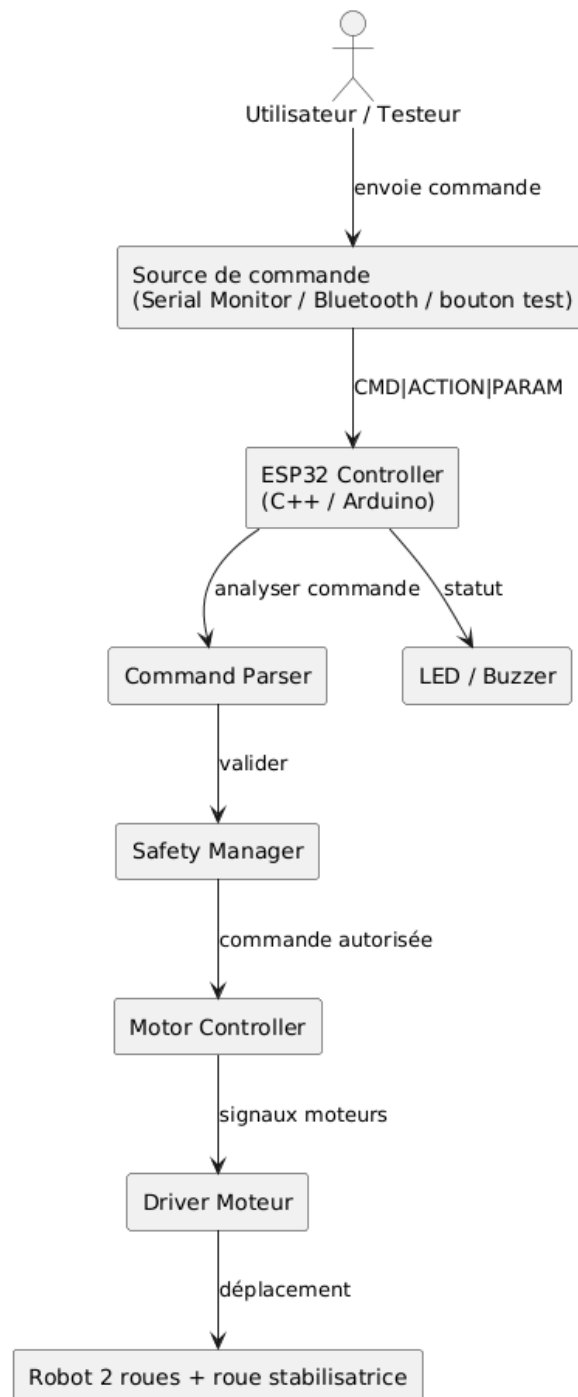


Figure 2 — Composants logiciels, phase 1

3.2. Phase 2 — Interface web embarquée

La phase 2 ajoute un serveur HTTP léger directement à l'intérieur de l'ESP32. Le code HTML, CSS et JavaScript de la page sont stockés en mémoire flash via le mécanisme PROGMEM. Le navigateur charge la page, l'utilisateur clique sur les boutons (Avancer ▲, Reculer ▼, Gauche ◀, Droite ▶, Stop ■) et chaque clic déclenche une requête HTTP GET sur l'URL /cmd avec les paramètres action, duration et speed. La requête est traduite en chaîne CMD|... et passée au même RobotController que celui de la phase 1, ce qui évite toute duplication de code.

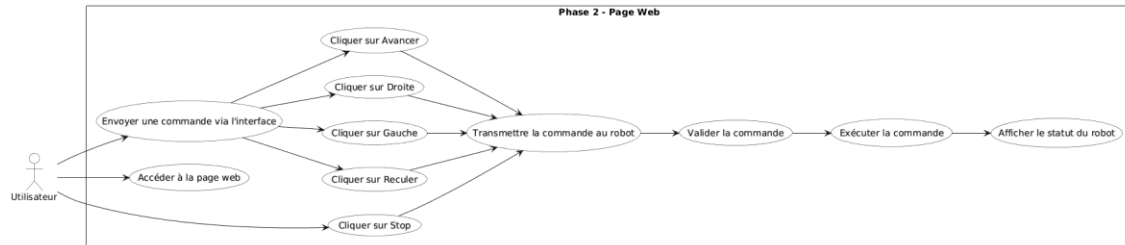


Figure 3 — Cas d'utilisation, phase 2

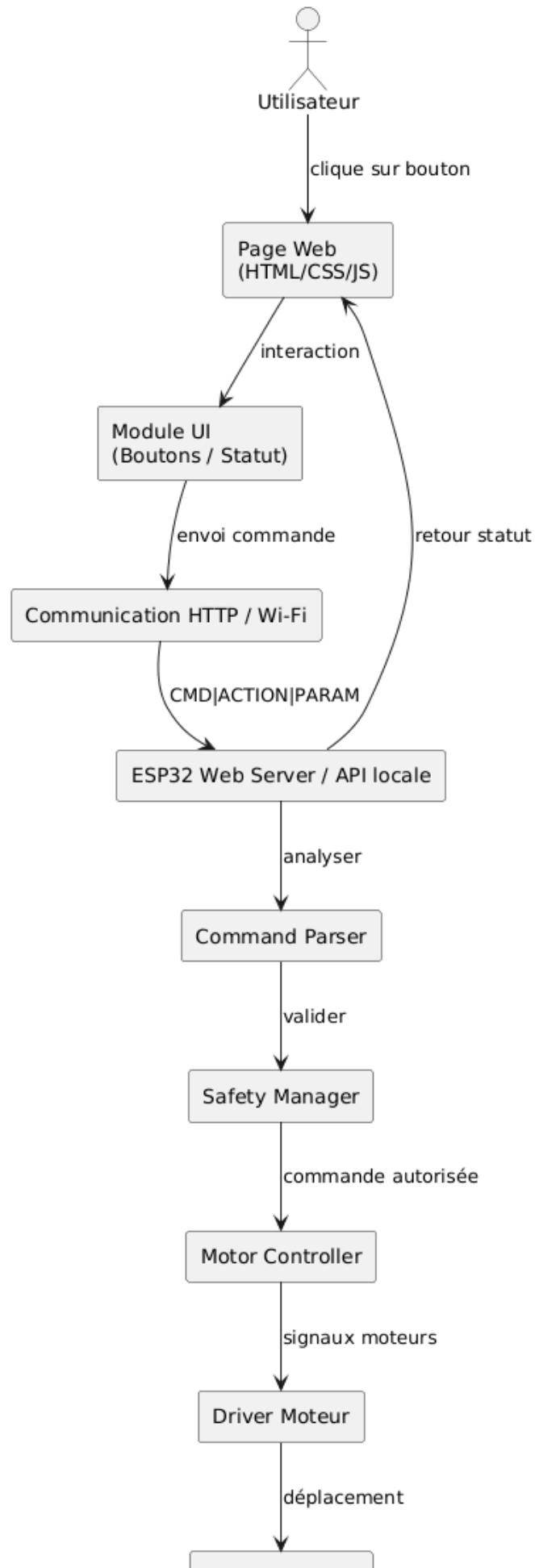


Figure 4 — Composants logiciels, phase 2

3.3. Phase 3 — Assistant vocal et chatbot

La phase 3 apporte la dimension « intelligente ». L'utilisateur parle dans le microphone du PC. Le texte transcrit (par speech_recognition côté Python ou par l'API Web Speech côté navigateur) est envoyé au serveur FastAPI qui décide s'il s'agit d'une commande pour le robot, d'une commande pour le téléphone Android, ou d'une simple discussion. Dans le dernier cas, le serveur interroge le modèle Ollama phi3:mini avec la persona « Luna » et renvoie une réponse courte en français.

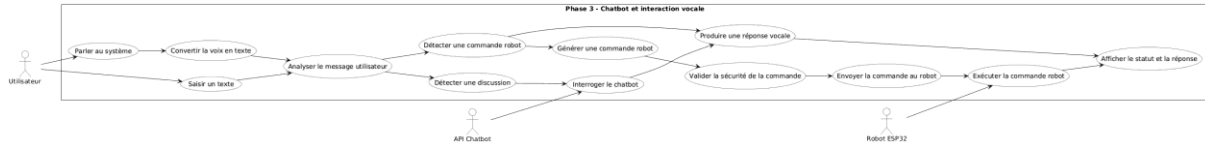


Figure 5 — Cas d'utilisation, phase 3

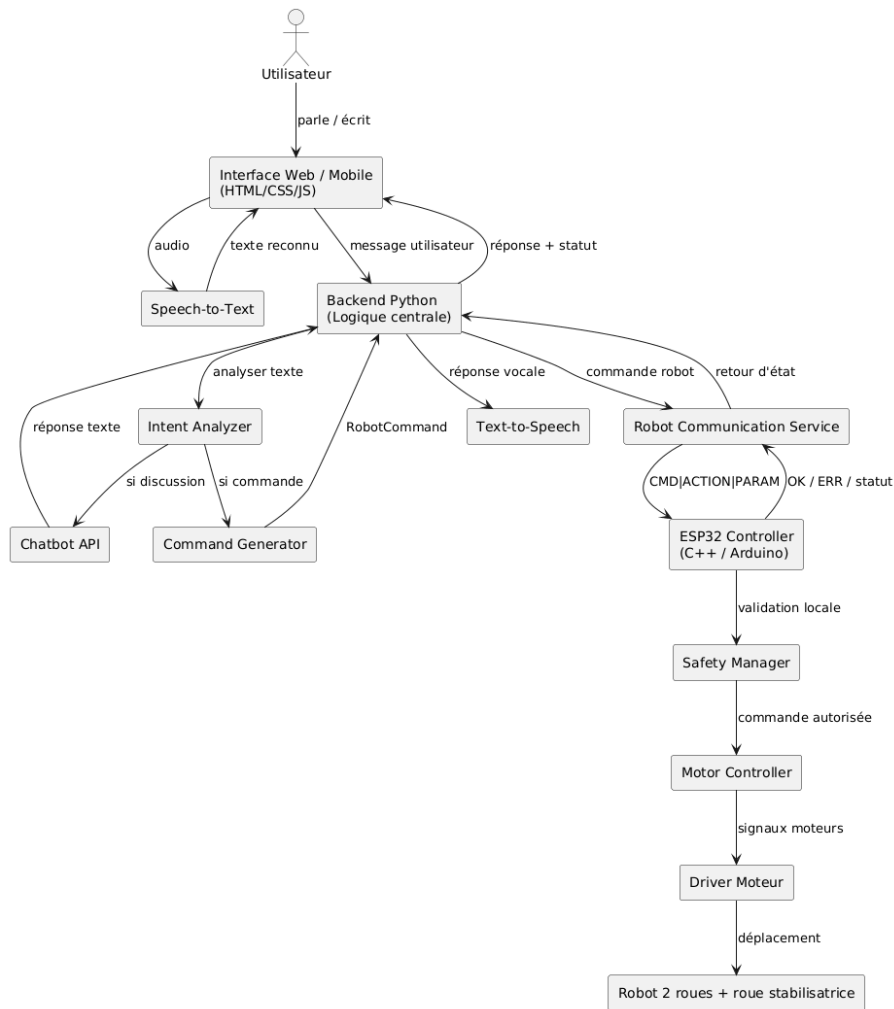


Figure 6 — Composants logiciels, phase 3

4. Matériel utilisé et brochage

Le robot est construit avec des composants courants et peu coûteux. Le tableau suivant détaille la liste du matériel et le rôle de chaque élément.

Composant	Rôle dans le projet
Carte ESP32 (ESP-WROOM-32)	Cerveau du robot. Lit les commandes, calcule les ordres moteurs (PWM) et héberge la page web.
Driver moteur L298N	Amplifie les signaux 3,3 V de l'ESP32 pour piloter les moteurs sous 7 à 12 V.
2 moteurs CC avec roues	Roues motrices gauche et droite. Permettent avance, recul et rotation sur place.
Roue folle (caster)	Stabilisatrice. Évite que le robot ne bascule en avant ou en arrière.
Châssis 2 roues	Support mécanique pour les moteurs, la batterie et les cartes.
Pack 4 à 6 piles AA (~9 V)	Alimente le L298N qui régule en interne le 5 V pour l'ESP32.
LED interne ESP32 (GPIO 2)	Indicateur visuel d'état (allumée quand le robot bouge).
Câbles Dupont	Connexions ESP32 ↔ L298N ↔ alimentation.
PC + microphone + haut-parleur	Pour la phase 3 (assistant vocal Python + Ollama).
Téléphone Android	Reçoit les commandes téléphone (appel, SMS, ouverture d'app) via NanoHTTPD.

Le brochage final retenu après l'évolution du projet est résumé ci-dessous. Toutes les broches sont regroupées sur un même côté du module ESP-WROOM-32, ce qui rend le câblage beaucoup plus lisible.

Broche ESP32	Connexion L298N	Constante dans le code
GPIO 26	IN1 (sens moteur gauche)	LEFT_MOTOR_IN1
GPIO 27	IN2 (sens moteur gauche)	LEFT_MOTOR_IN2
GPIO 25	ENA (vitesse moteur gauche)	LEFT_MOTOR_ENA
GPIO 14	IN3 (sens moteur droit)	RIGHT_MOTOR_IN1
GPIO 33	IN4 (sens moteur droit)	RIGHT_MOTOR_IN2
GPIO 32	ENB (vitesse moteur droit)	RIGHT_MOTOR_ENB
GPIO 2	LED interne d'état	STATUS_LED
GND	GND commun avec L298N + batterie	—

Schema de cablage - ESP32 + L298N + 2 moteurs

Lire de gauche a droite. Chaque fil est etiquete avec sa fonction.

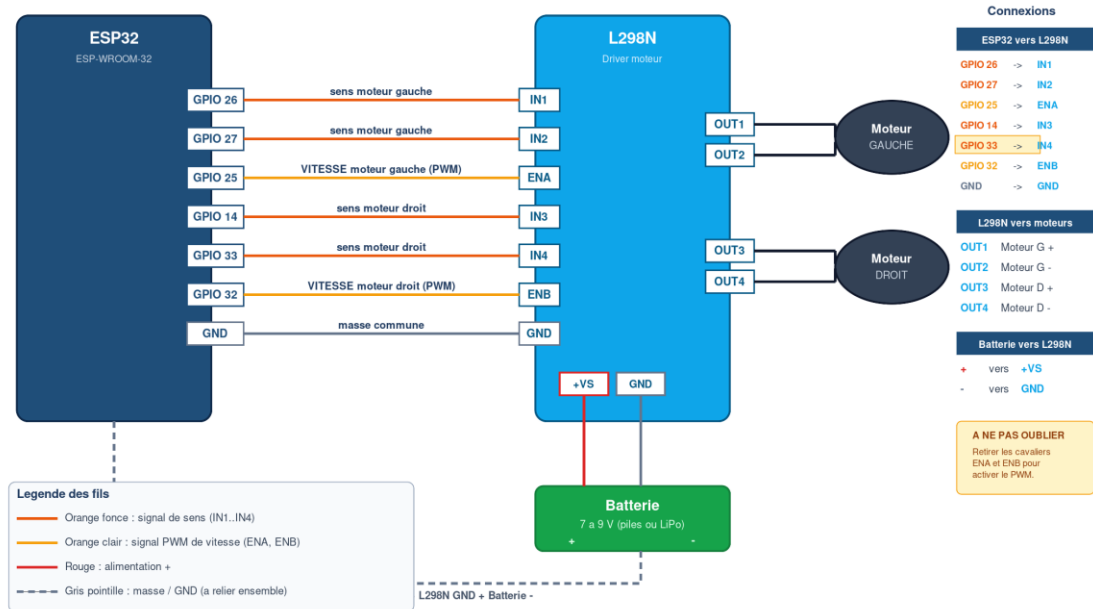


Figure 7 — Schéma de câblage complet ESP32 + L298N + moteurs

5. Présentation du code

Le code est volontairement découpé en plusieurs classes courtes, chacune responsable d'une seule chose. Cette approche, inspirée du principe de responsabilité unique, rend le programme facile à lire, à tester et à faire évoluer.

5.1. Sketch Arduino (ESP32)

Le fichier `robot_mobile_intelligent.ino` contient cinq classes qui collaborent ensemble.

Classe	Responsabilité
MotorController	Pilote sens (IN) et vitesse (PWM via ledc) des deux moteurs.
SafetyManager	Borne la durée et la vitesse à des limites de sécurité.
CommandParser	Découpe la chaîne <code>CMD ACTION DUREE VITESSE</code> en structure exploitable.
RobotController	Chef d'orchestre : reçoit, valide, exécute, arrête automatiquement.
WebInterface	Connecte le Wi-Fi, démarre le serveur HTTP, sert HTML/CSS/JS et <code>/cmd</code> .

L'API PWM utilisée est la nouvelle version simplifiée du core ESP32 v3.x : `ledcAttach(broche, fréquence, résolution)` qui combine en une seule fonction l'ancien duo `ledcSetup + ledcAttachPin`. La page web (HTML, CSS, JavaScript) est embarquée dans trois chaînes `PROGMEM` pour économiser la RAM. La page utilise `fetch()` pour appeler `/cmd` avec les paramètres, et un ping toutes les 5 s vérifie que l'ESP32 est toujours joignable (voyant vert/rouge en haut de la page).

5.2. Page de contrôle web

L'interface est conçue mobile-first avec un thème sombre type tableau de bord. Elle propose un pavé directionnel en croix (5 boutons), un curseur de vitesse (80–255), un curseur de durée (100–2000 ms), une console temps réel des commandes envoyées et reçues, et un indicateur de connexion. Le pilotage est également possible au clavier (flèches + barre espace pour Stop).

Une seconde page `commande_vocale.html` utilise l'API Web Speech de Chrome pour reconnaître la voix de l'utilisateur en français, analyser localement la phrase (avance / recule / gauche / droite / stop, durée en secondes, vitesse rapide/lente) et envoyer la commande à l'ESP32, le tout sans dépendre du backend Python.

5.3. Backend Python + Ollama

Le backend FastAPI expose une route `POST /process` qui reçoit le texte transcrit par `speech_recognition`, décide via la fonction `decide()` s'il s'agit d'une commande robot, d'une commande téléphone ou d'une discussion, puis route vers le bon traitement. Pour la discussion, le module Ollama est interrogé avec la persona « Luna » (chatbot amical en français).

6. Difficultés rencontrées

Plusieurs difficultés techniques ont rythmé le projet, principalement côté matériel.

- Mise à jour du driver USB-série CH340 incompatible avec Windows 11 : l'ancienne version 3.4 (2014) refusait de s'installer.
- Le bouton BOOT de la carte ESP-WROOM-32 ne répondait pas systématiquement, ce qui empêchait le passage en mode flash.
- Erreurs « 2-sync fail » et « Write timeout » répétitives lors du téléversement via esptool et Flash Download Tool.
- Évolution de l'API PWM entre les versions 2.x et 3.x du core ESP32 (ledcSetup → ledcAttach) provoquant des erreurs de compilation.
- Un court-circuit accidentel lors du test du fil GPIO 0 ↔ GND a temporairement coupé la carte.
- La reconnaissance vocale Google Speech (online) demande une bonne connexion Internet et n'est pas idéale pour la démo.
- Le modèle phi3:mini d'Ollama mettait plusieurs secondes à répondre la première fois (chargement en RAM).
- Le réseau Wi-Fi domestique isolait parfois le PC et l'ESP32 sur des sous-réseaux différents.

7. Solutions apportées

Difficulté	Solution adoptée
Driver CH340 refuse de s'installer	Téléchargement de la version 3.7 signée + exécution en mode administrateur.
Bouton BOOT capricieux	Procédure manuelle : maintenir BOOT, tapoter EN, relâcher BOOT, puis cliquer Téléverser.
Write timeout / 2-sync fail	Réduction de la vitesse à 115200 bauds, fermeture stricte de tous les outils tenant COM4.
Erreur ledcSetup non déclaré	Adaptation à la nouvelle API ESP32 v3.x : ledcAttach(broche, freq, resolution).
Court-circuit accidentel	Repos de 10 minutes pour reset du polyfuse, puis recablage propre, USB en dernier.
Reconnaissance vocale dépendante d'Internet	Préparation d'une alternative Web Speech API + envisagé Vosk pour usage hors-ligne.
Ollama lent au premier appel	Lancement préventif de ollama serve avant la démo, premier prompt de chauffe.
Wi-Fi isolant les appareils	Création d'un hotspot téléphone (Robot_mamadou / Number1234) pour regrouper tous les appareils.

8. Résultats obtenus

Le robot final répond aux trois phases prévues au cahier des charges. Le code Arduino occupe 73 % de la flash et 14 % de la RAM, ce qui laisse une grosse marge pour des extensions futures.

- Phase 1 — pilotage par moniteur série au format CMD|ACTION|DUREE_MS|VITESSE : fonctionnel.

- Phase 2 — page web embarquée accessible depuis téléphone ou PC, avec retour en temps réel : fonctionnel.
- Phase 3 — assistant vocal en français routé par FastAPI vers le robot, le téléphone ou le chatbot Luna : fonctionnel.
- Sécurité matérielle : la durée et la vitesse sont systématiquement bornées avant exécution.
- Téléversement final réussi après résolution du problème de driver et de la procédure BOOT.

9. Améliorations possibles

- Ajouter un capteur à ultrasons HC-SR04 pour l'évitement d'obstacles en temps réel.
- Encoder les roues pour réaliser des rotations à angle précis (90°, 180°).
- Sécuriser la page web (mot de passe ou token) pour éviter qu'un tiers du même Wi-Fi pilote le robot.
- Remplacer Google Speech (en ligne) par Vosk pour une reconnaissance vocale hors-ligne.
- Utiliser une synthèse vocale neuronale (Coqui TTS, Piper) pour donner une vraie voix à Luna.
- Ajouter une caméra ESP32-CAM pour voir en direct ce que voit le robot.
- Imprimer un châssis personnalisé en 3D pour intégrer proprement la batterie, le L298N et l'ESP32.
- Mettre en place une mise à jour OTA pour reflasher l'ESP32 sans câble USB.

10. Conclusion

Ce projet a permis de mettre en pratique de nombreuses notions vues en cours : programmation embarquée, électronique de puissance, communication série, programmation web (HTML, CSS, JavaScript), programmation orientée objet en C++, API REST avec FastAPI, et intégration concrète d'un modèle de langage local. L'approche en trois phases successives s'est révélée payante : chaque couche a pu être validée séparément, ce qui a évité de chercher en même temps un bug mécanique, logiciel et réseau.

Les difficultés rencontrées, notamment côté matériel (drivers, bouton BOOT capricieux, téléversement instable), ont été l'occasion d'apprendre à diagnostiquer méthodiquement un problème, à utiliser plusieurs outils différents (Arduino IDE, Flash Download Tool, esptool en ligne de commande) et à ne pas baisser les bras face à un message d'erreur cryptique.

Au-delà de l'aspect technique, le projet a aussi été l'occasion de travailler la rigueur : nommer clairement les classes, commenter le code, séparer les responsabilités, borner systématiquement les valeurs reçues de l'extérieur. Ce sont autant de réflexes qui resserviront dans des projets futurs plus complexes.

Avec les améliorations proposées (capteur d'obstacles, encoder, caméra, sécurité de la page web), le robot peut évoluer vers un véritable petit véhicule autonome capable de cartographier son environnement et de dialoguer naturellement avec son utilisateur.